

Informe de Auditoría del Sistema Web

Integrantes del grupo:

Jorge Illera Rivera

Iker Argulo Galán

Oscar Lijerón Pérez

Bruno Izaguirre Martínez de Marañón

María Bogajo

Ander Javier Corral

Fecha:

09/11/2024

Índice de Contenidos

1	Introducción	3
2	Metodología	3
3	Vulnerabilidades	3
3.1	Rotura de control de acceso	3
3.1.1	Soluciones implementadas	3
3.1.2	Soluciones implementadas	3
3.2	Fallos criptográficos	4
3.2.1	Soluciones implementadas	4
3.3	Inyección	5
3.3.1	Cross-Site Scripting (XSS)	6
3.3.2	Inyección SQL	6
3.3.3	Soluciones implementadas	7
3.4	Diseño inseguro	8
3.4.1	Soluciones implementadas	8
3.5	Configuración de seguridad insuficiente	11
3.5.1	Soluciones implementadas	11
3.6	Componentes vulnerables y obsoletos	11
3.6.1	Soluciones implementadas	11
3.7	Fallos de identificación y autenticación	11
3.7.1	Soluciones implementadas	11
3.8	Fallos en la integridad de datos y software	12
3.8.1	Soluciones implementadas	12
3.9	Fallos en la monitorización de la seguridad	12
3.9.1	Soluciones implementadas	12
4	Conclusiones	12
5	Bibliografía	12
6	Apéndices	13

Índice de Figuras

1	Errores de inyección detectados por ZAP.	6
2	Desplazamiento del botón Juegos	10

Índice de Tablas

1 Introducción

Este informe detalla la auditoría de seguridad del sistema web utilizando OWASP ZAP, con el objetivo de identificar y mitigar vulnerabilidades.

2 Metodología

La auditoría se realizó mediante un análisis de ZAP, ejecutando...

3 Vulnerabilidades

3.1 Rotura de control de acceso

Una rotura del control de acceso ocurre cuando un sistema permite a usuarios acceder a datos, funciones o recursos para los que no tienen autorización, debido a fallos en la implementación de los permisos. Esto puede resultar en accesos no autorizados, modificaciones indebidas de datos o acciones privilegiadas que deberían estar restringidas.

3.1.1 Soluciones implementadas

Para minimizar la utilización de **CORS**¹, se ha integrado el uso de tokens **CSRF**² con el fin de implementar mecanismos de acceso únicos.

Una rotura del control de acceso ocurre cuando un sistema permite a usuarios acceder a datos, funciones o recursos para los que no tienen autorización, debido a fallos en la implementación de los permisos. Esto puede resultar en accesos no autorizados, modificaciones indebidas de datos o acciones privilegiadas que deberían estar restringidas.

3.1.2 Soluciones implementadas

Para minimizar la utilización de **CORS**³, se ha integrado el uso de tokens⁴ anti **CSRF**⁵ con el fin de implementar mecanismos de acceso únicos. **Integración de tokens CSRF:**

Se ha implementado la creación de los tokens en archivos donde se manejan formularios o se ingresan los identificadores y contraseñas de los usuarios. Esto incluye archivos como , , y .

¹Mecanismo que permite que los recursos de un servidor sean accesibles desde diferentes dominios de manera controlada.

²Códigos únicos generados para verificar que una solicitud proviene de una fuente legítima y no de un atacante.

³Mecanismo que permite que los recursos de un servidor sean accesibles desde diferentes dominios de manera controlada.

⁴Valor único y aleatorio generado por un servidor y asociado a la sesión de un usuario, que se envía junto con los formularios o solicitudes sensibles.

⁵Códigos únicos generados para verificar que una solicitud proviene de una fuente legítima y no de un atacante.

- 1. **Generación del token CSRF:**
Si no existe ya un token CSRF en la sesión , se genera uno nuevo usando . Con esto se asegura que cada sesión tiene un token único y difícil de adivinar.
- 2. **Uso del token en el formulario:**
En el formulario HTML⁶ (HyperText Markup Language)(), este token se debe enviar como un campo oculto. Esto permite que el servidor valide que el formulario fue enviado desde la misma aplicación.
- 3. **Validación del token en el servidor:**
Cuando el formulario se envía, el código verifica si el token recibido () coincide con el que está almacenado en la sesión. Si no coinciden o el token no existe, la solicitud se considera no válida y se bloquea ().
- 4. **Eliminación del token:**
Una vez que el usuario inicia sesión correctamente, el token se elimina usando el comando (). Esto asegura que no pueda ser reutilizado.

3.2 Fallos criptográficos

En esta sección se comentarán los fallos de seguridad web relacionados con la criptografía⁷ (o la falta de esta) causando la exposición de datos sensibles[8].

Se han identificado y solucionado dos fallos criptográficos. El primer fallo consiste en almacenar la información de la base de datos sin cifrar, utilizando únicamente el cifrado incorporado por SQL, el cual se descifra directamente al realizar una consulta. El segundo fallo detectado es almacenar las contraseñas en texto plano.

3.2.1 Soluciones implementadas

Para subsanar el primer error se ha decidido cifrar la información que se almacena en la base de datos. El algoritmo de cifrado simétrico⁸ que se ha usado es el AES (Advanced Encryption Standar) de 256 bits de longitud, puesto que este algoritmo es considerado seguro a día de hoy[10].

El uso de este algoritmo requiere una clave segura para cifrar y descifrar los datos. Para ello se ha usado una función[5] de Python⁹ que genera la clave usando la librería , que es un módulo que genera números aleatorios con la suficiente entropía para que la clave generada pueda ser considerada criptográficamente fuerte. Una vez generada la clave de secreta se guarda como una variable de entorno¹⁰ en un archivo .env llamado para evitar

⁶Lenguaje de marcado utilizado para estructurar y organizar el contenido en las páginas web.

⁷Práctica que consiste en proteger información mediante el uso de algoritmos codificados, Hashes y firmas.

⁸Método de cifrado que usa la misma clave para cifrar y descifrar.

⁹Lenguaje de programación orientado a objetos usado principalmente para el desarrollo web y de aplicaciones informáticas.

¹⁰Las variables de entorno son variables disponibles para tu programa/aplicación de forma dinámica durante el tiempo de ejecución. El valor de estas variables puede proceder de diversas fuentes: archivos de texto, gestores secretos de terceros, scripts de llamada, etc.[6]

comprometerla. Los datos cifrados de la tabla Juegos son el nombre, género, fecha de lanzamiento, valoración y precio. Mientras que los datos cifrados de la tabla Usuarios son el nombre, teléfono, DNI y fecha de nacimiento. Sin embargo, como al cifrar los datos su número de caracteres aumenta, se han tenido que cambiar los tipos de dato y la longitud de los campos de la base de datos para poder almacenarlos.

Para solventar el segundo error se ha decidido usar un algoritmo resumen para generar el resumen criptográfico o Hash¹¹ de la contraseña en vez de guardarla como texto plano. Para ello se ha usado la función `sha1` a la que se le pasa como parámetros la contraseña a resumir y una constante de PHP¹² (Hypertext Preprocessor) que indica que queremos usar el algoritmo Bcrypt[7]. Bcrypt usa sales¹³ para mayor seguridad ya que hace que dos contraseñas iguales de usuarios diferentes generen resúmenes criptográficos distintos, por lo que se hace mucho mas difícil precalcularlos.

3.3 Inyección

¿Qué es la inyección?

La inyección es una vulnerabilidad de seguridad en la que un atacante introduce datos maliciosos en una aplicación, a menudo mediante campos de entrada o URL, logrando manipular comandos o consultas dentro del sistema. Esto permite ejecutar código no autorizado, acceder a datos sensibles o modificar información sin permisos[1].

En esta sección se han identificado dos amenaza de alto riesgo. Las vulnerabilidades detectadas corresponden a una Inyección SQL y a un ataque de Cross-Site Scripting (XSS) Reflejado, como se puede observar en la Figura 2.

¹¹Criptograma que consiste en una sucesión de caracteres alfanuméricos que representan la totalidad del contenido original.

¹²PHP es un lenguaje de código abierto especialmente adecuado para el desarrollo web y que puede ser incrustado en HTML.[4]

¹³Caracteres aleatorios que se le añaden a la contraseña para que sea mas difícil precalcular los resúmenes criptográficos.



Figura 1: Errores de inyección detectados por ZAP.

3.3.1 Cross-Site Scripting (XSS)

La vulnerabilidad CWE-79 corresponde a Cross-Site Scripting (XSS) Reflejado, que permite a un atacante inyectar código malicioso en los parámetros de entrada del sistema, como los campos de formularios o las URL. Cuando un usuario accede a una URL o un formulario que contiene este código inyectado, el navegador del usuario ejecuta dicho código, lo que puede llevar a la manipulación de datos, el robo de cookies, la ejecución de scripts no autorizados, y otros ataques de seguridad.[2]

Causa de la vulnerabilidad

La vulnerabilidad XSS detectada se originaba en el archivo , más concretamente en el campo de la fecha de lanzamiento y en otros campos de entrada en el formulario de modificación de un juego. Esto se debía a que los valores de entrada eran reflejados en la respuesta sin la sanitización o validación adecuada, permitiendo a los atacantes inyectar y ejecutar código JavaScript en el navegador del usuario.

Pruebas y comportamiento observado

Sin la sanitización de entrada: OWASP ZAP detectó que ciertos campos permitían la inyección de código JavaScript en los valores de los formularios.

Por ejemplo, en el campo de la fecha, se podía ingresar un valor como , lo cual provocaba la ejecución de un alert en el navegador.

3.3.2 Inyección SQL

La vulnerabilidad CWE-89 corresponde a la inyección SQL, donde un atacante es capaz de insertar o manipular consultas SQL a través de los datos de entrada, en este caso, el parámetro en la URL.

Causa de la Vulnerabilidad

La vulnerabilidad detectada se originaba en el archivo `index.php` y fue causada por la construcción dinámica de consultas SQL sin la adecuada sanitización¹⁴ ni parametrización de los datos proporcionados por el usuario. En este caso, el valor del parámetro `id` se concatenó directamente a la consulta SQL sin protección, lo que permitió la ejecución de código SQL malicioso.

Pruebas y comportamiento observado

En el error encontrado, la inyección fue posible al modificar el parámetro de la siguiente manera:

Esto permitió a OWASP ZAP modificar la consulta SQL ejecutada en el servidor, manipulando los resultados devueltos y accediendo a información que no debía ser revelada. En particular, se manipuló la consulta para obtener datos utilizando una condición booleana que siempre era verdadera, `1=1`, y luego se comentó el resto de la consulta utilizando `--`, lo que invalidaba el resto de la consulta SQL.

3.3.3 Soluciones implementadas

Para mitigar la vulnerabilidad de XSS se implementaron las siguientes soluciones:

- **Escapado¹⁵ de salida:**
Se utilizó la función `htmlspecialchars` en PHP para escapar caracteres especiales (como `<`, `>`, `&`, `"`, `'`) en todas las salidas. Esto asegura que el contenido reflejado en el navegador sea tratado como texto y no como código ejecutable.
- **Validación y sanitización de fecha de lanzamiento:**
Se implementó una expresión regular `/^\d{4}-\d{2}-\d{2}$/` para asegurarse de que las fechas ingresadas cumplan con el formato `YYYY-MM-DD`.
- **Sanitización de datos entrantes:**
Para cada valor de entrada recibido a través del formulario, se aplicó `htmlspecialchars` para evitar que cualquier código especial o malicioso llegue a la base de datos.
- **Validación de Campos Numéricos (rating y price):**
Los valores de rating y price se convirtieron a tipo float usando `floatval` para asegurarse de que se almacenen y procesen correctamente como valores numéricos.

Para mitigar la vulnerabilidad de Inyección SQL se implementaron las siguientes soluciones:

- **Uso de consultas preparadas:**
Se ha utilizado el enfoque de consultas preparadas con declaraciones `PreparedStatement`. Esto asegura que los parámetros sean tratados de forma segura, separando los datos del código

¹⁴Sanitización se refiere al proceso de limpiar o filtrar los datos de entrada para eliminar cualquier contenido malicioso que pudiera comprometer la seguridad del sistema.[3]

¹⁵Escapado se refiere al proceso de transformar caracteres especiales en una entrada válida para que no sean interpretados como parte del código ejecutable.[3]

SQL, y evitando así que cualquier entrada maliciosa pueda modificar la estructura de las consultas.

- **Validación y sanitización de entradas**

Además de las consultas preparadas, se valida y filtra las entradas del usuario para asegurarse de que solo se acepten datos válidos. Por ejemplo, el uso de `parseInt` para el parámetro `id` convierte cualquier valor no numérico a un valor seguro (como 0), evitando que se inyecten datos inesperados en las consultas.

Todas estas soluciones también se han implementado en otros archivos que, aunque no presentaban errores en el análisis, tenían vulnerabilidades relacionadas con la inyección que necesitaban ser abordadas.

Resultados de la segunda auditoría

Después de implementar las soluciones descritas, se realizó un segundo análisis con OWASP ZAP. Los resultados fueron positivos: no se detectaron vulnerabilidades de XSS ni otros problemas de inyección.

Conclusiones

Ambas vulnerabilidades fueron una amenaza crítica para la aplicación, ya que podía permitir a los atacantes ejecutar código malicioso en el navegador de los usuarios o insertar consultas SQL a través de los datos de entrada. Gracias a las soluciones implementadas, las vulnerabilidades se han mitigado de manera efectiva. La aplicación ahora cumple con un estándar de seguridad más alto, protegiendo los datos y la experiencia de los usuarios frente a posibles ataques de inyección de código.

3.4 Diseño inseguro

¿Que significa diseño inseguro?

Una página web tiene un diseño inseguro cuando su estructura, funcionalidades¹⁶, o configuración comprometen la seguridad de los usuarios o de sus datos. Esto puede deberse a malas prácticas en el desarrollo, falta de medidas de seguridad adecuadas o errores en la implementación.

En esta sección se ha identificado una vulnerabilidad principal. La no validación de los datos introducidos por los usuarios al registrarse. Además, se ha desplazado la posibilidad de ver, modificar y añadir videojuegos a la plataforma, exclusivamente para usuarios registrados.

3.4.1 Soluciones implementadas

Para mitigar las vulnerabilidades de la no validación de datos, se han implementado las siguientes soluciones:

¹⁶Características y herramientas que permiten a los usuarios interactuar con el sitio para cumplir objetivos.

– **Validación de campos:**

En esta sección se comprueba que el usuario haya rellenado todos los campos obligatorios del formulario¹⁷. Para ello se ha utilizado una función: . Para comprobar que todos los campos han sido rellenados correctamente.

– **Validación de nombre:**

En esta sección se comprueba que el usuario haya introducido un nombre: Para ello se ha utilizado una función: . Con esta función se comprueba que el nombre introducido es un nombre real formado únicamente por letras y espacios.

– **Validación de DNI:**

En esta sección se comprobará que el usuario introduzca un DNI válido, formado por:

1. 8 numeros.
2. Un guión.
3. Una letra.

Para ello se ha utilizado una función: .

– **Validación de email:**

En esta sección se verifica que el email que ha introducido el usuario sea válido. Para ello se ha utilizado una función: . Con esta función se consigue verificar que el email introducido está formado por las siguientes partes:

1. Cadena de caracteres, representando el nombre de usuario.
2. Una arroba¹⁸.
3. Cadena de caracteres reflejando el dominio.
4. Un '.' .
5. Un TLD¹⁹(Top-Level Domain) de al menos dos letras.

– **Validación de teléfono:**

En esta sección se comprobará que el usuario introduzca un teléfono válido de exactamente 9 dígitos.

– **Validación de fecha de nacimiento:**

En esta sección se verificará que la fecha de nacimiento del usuario es válida, es decir, no puede ser posterior a la fecha actual.

– **Validación de contraseña:**

En esta sección se verificará que la contraseña introducida por el usuario cumple unos requisitos mínimos de seguridad, utilizando la siguiente función:

. Los requisitos mínimos impuestos son los siguientes:

1. Mínimo de 8 caracteres.
2. Al menos una letra mayúscula.

¹⁷Una interfaz interactiva que permite a los usuarios ingresar y enviar información estructurada, la cual se procesa y almacena en la base de datos correspondiente.

¹⁸La arroba (@) es un símbolo utilizado principalmente para separar el nombre del usuario y el dominio en una dirección de correo electrónico.

¹⁹Es la parte final de un nombre de dominio en Internet, que identifica la categoría, propósito o ubicación geográfica del dominio

3. Al menos un número.

Desplazamiento del botón para ver/añadir/modificar juegos:

Se ha desplazado el botón de la pantalla principal que aparece nada más acceder a la web a una pantalla posterior después de haber iniciado sesión, esto se puede apreciar en la Figura 2. Con esto se consigue evitar que usuarios sin registrar puedan llegar a saturar la **BBDD**²⁰(Base de datos) y que esta deje de responder.

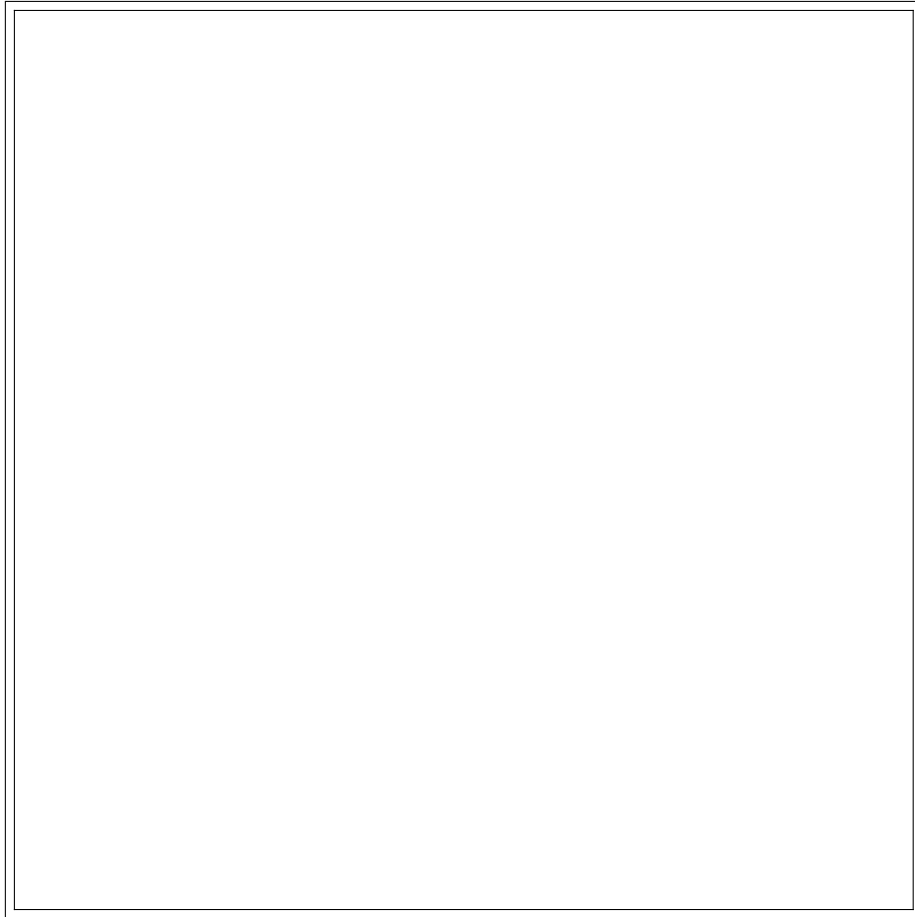


Figura 2: Desplazamiento del botón **Juegos**.

²⁰Sistema estructurado para almacenar y gestionar datos, permitiendo su acceso y modificación de manera eficiente y segura.

3.5 Configuración de seguridad insuficiente

3.5.1 Soluciones implementadas

3.6 Componentes vulnerables y obsoletos

3.6.1 Soluciones implementadas

3.7 Fallos de identificación y autenticación

¿Qué es un fallo de identificación y autenticación?

Un fallo de identificación ocurre cuando el sistema no reconoce quién dices ser. Esto puede deberse a varias razones, como ingresar datos incorrectos o inexistentes, que el sistema no busque correctamente el usuario en la base de datos, o incluso por el uso de mensajes de error ambiguos o confusos que no ayudan al usuario a identificar el problema. Además, la ausencia de mensajes claros también puede contribuir a que el sistema no reconozca correctamente al usuario.

Un fallo de autenticación ocurre cuando el sistema no puede verificar que eres quién dices ser. Esto puede suceder porque el usuario ha registrado una contraseña insegura o fácil de adivinar, utiliza mecanismos simples para recuperar contraseñas, o bien porque no se ha implementado un sistema de autenticación más robusto como la autenticación multifactor²¹.

¿Cómo ocurre un ataque por fallos de identificación o autenticación?

Esta vulnerabilidad se da cuando un atacante obtiene una lista de usuarios y contraseñas que pueden ser válidas en el sistema, y luego intenta acceder a las cuentas mediante un proceso automatizado. Un ejemplo común es el ataque de *credential stuffing*²². En este ataque, el atacante utiliza credenciales filtradas de otros servicios donde los usuarios han utilizado las mismas contraseñas, y las prueba en el sistema objetivo para obtener acceso a sus cuentas. Si el sistema no implementa medidas adicionales de seguridad, como la autenticación multifactor o límites de intentos de acceso fallidos, el atacante puede obtener acceso a las cuentas de usuario con relativa facilidad.

3.7.1 Soluciones implementadas

Para reducir los riesgos de los errores en la identificación y autenticación se ha implementado la siguientes soluciones:

²¹Método de autenticación que requiere que el usuario proporcione dos o más factores de verificación diferentes para acceder a un sistema

²²Ataque cibernético en el que los atacantes usan combinaciones de nombres de usuario y contraseñas robadas de otras plataformas para intentar acceder a cuentas en diferentes sitios web o servicios.

– **Registro de todos los intentos de sesiones:**

Se ha utilizado un archivo `.log`²³ para poder gestionar y observar el por qué de los errores de dichos intentos de sesiones. En el archivo `.log` se registra lo siguiente:

1. Fecha y hora en formato `yyyy-mm-dd hh:mm:ss`.
2. `SUCCESS` o `FAILED`.
3. Email registrado o no en la base de datos.
4. `Ip`²⁴.

– **Bloqueo temporal de usuarios por logins²⁵ fallidos:**

Con ayuda del archivo `.log` se consigue registrar aquellos usuarios que han fallado 5 veces en los últimos 10 minutos. Dicha cuenta se bloquea por 10 minutos y si el usuario intenta hacer nuevamente un login, se le mostrará el tiempo que le falta para que pueda volver a realizar otros 5 intentos de login. Esto se logra gestionar en el archivo `.log` con la función:

3.8 Fallos en la integridad de datos y software

3.8.1 Soluciones implementadas

3.9 Fallos en la monitorización de la seguridad

3.9.1 Soluciones implementadas

4 Conclusiones

5 Bibliografía

References

- [1] owasp.org
Injection
(accedido 11/07/2024).
- [2] owasp.org
Cross-Site Scripting (XSS)
(accedido 11/07/2024).
- [3] cwe.mitre.org
Sanitización y escapeo
(accedido 11/07/2024).

²³archivo de texto que se utiliza para registrar eventos, actividades o mensajes de un sistema, programa o aplicación

²⁴Identificador único que se asigna a cada dispositivo conectado a una red

²⁵Proceso mediante el cual un usuario se identifica y autentica en un sistema, servicio o aplicación.

- [4] [php.net](#)
¿Qué es PHP?
(accedido 11/07/2024).
- [5] Python Code: Generación de Claves de Cifrado
Este código en Python se utiliza para generar claves criptográficamente seguras usando la librería `base64` basado en código generado por ChatGPT.



- [6] [kinsta.com](#)
¿Que es una variable de entorno?
(accedido 15/11/2024).
- [7] [skysnag.com](#)
¿Que es Bcrypt?
(accedido 15/11/2024).
- [8] [owasp.org](#)
¿Que son los fallos criptográficos?
(accedido 16/11/2024).
- [9] [aws.amazon.com](#)
¿Que es la criptografía?
(accedido 16/11/2024).
- [10] [msmk.university](#)
¿Qué es AES? ¿Es seguro?
(accedido 16/11/2024).

6 Apéndices